Real-world (rebuild Doc04's extraction, LCEL-style) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `lcel_vs_raw_sdk_practice.py`: reading about "LangChain vs. raw SDK" isn't the same as running both versions of one real feature side by side. This rebuilds Doc04's `get_structured_reply()` with LCEL and compares the two. **If not:** the Build Task's `compare_with_raw_sdk.py` would be the first time you built this comparison, with no smaller version to trust it against.

All examples below need Doc04's `chat_client.py` and `schemas.py` copied, unchanged, into `practice/` next to this file, and are run from inside `practice/`.

Basic Version

Approach 1 — print and eyeball

```
# lcel_vs_raw_sdk_practice.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

from chat_client import EXTRACT_PROMPT, create_client, get_structured_reply
from schemas import ExtractedInfo

# how: the same two messages Doc04's raw call sends — the system rules,
# then the user's text in a {text} slot
prompt = ChatPromptTemplate.from_messages(
    [("system", EXTRACT_PROMPT), ("human", "{text}")]
)
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
lcel_chain = prompt | model.with_structured_output(ExtractedInfo)

client = create_client()
test_inputs = [
    "Maria wants to reset her password",
    "Hi, I'm Tom and my invoice looks wrong",
    "Ahmed here, please cancel my order",
]
```

```
for text in test_inputs:
    raw_result = get_structured_reply(client, text)
    lcel_result = lcel_chain.invoke({"text": text})
    print("Input:", text)
    print("  raw: ", raw_result)
    print("  lcel:", lcel_result)
```

Expected output (wording varies):

```
Input: Maria wants to reset her password
  raw:  name='Maria' intent='reset her password'
  lcel: name='Maria' intent='reset her password'
Input: Hi, I'm Tom and my invoice looks wrong
  raw:  name='Tom' intent='check a wrong invoice'
  lcel: name='Tom' intent='check a wrong invoice'
Input: Ahmed here, please cancel my order
  raw:  name='Ahmed' intent='cancel his order'
  lcel: name='Ahmed' intent='cancel his order'
```

This shows both results side by side, but leaves the comparing to your eyes — easy to get wrong past a few inputs.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — compute the match

Story — `compare()`: the script decides "match" itself, with a fair rule: names must agree; intent wording may differ. **If not:** a human reads printed text and misses a real difference.

```
# lcel_vs_raw_sdk_practice.py
from openai import OpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import Runnable
from langchain_openai import ChatOpenAI

from chat_client import EXTRACT_PROMPT, create_client, get_structured_reply
from schemas import ExtractedInfo

# how: every input has a name, so neither version returns None here
TEST_INPUTS = [
    "Maria wants to reset her password",
    "Hi, I'm Tom and my invoice looks wrong",
    "Ahmed here, please cancel my order",
]


def build_lcel_chain(model_name: str = "gpt-4o-mini") -> Runnable:
    prompt = ChatPromptTemplate.from_messages(
        [("system", EXTRACT_PROMPT), ("human", "{text}")]
    )
    model = ChatOpenAI(model=model_name, temperature=0)
    return prompt | model.with_structured_output(ExtractedInfo)


def compare(client: OpenAI, lcel_chain: Runnable, text: str) -> bool:
    raw_result = get_structured_reply(client, text)
    lcel_result = lcel_chain.invoke({"text": text})
    # why: a name has one right answer, so both versions must agree on
    # it; intent is free wording that can differ between any two calls
    matched = raw_result.name == lcel_result.name
    print(f"{text} -> match: {matched}")
    print(f"  raw:  {raw_result}")
    print(f"  lcel: {lcel_result}")
```

```
    return matched


def main() -> None:
    client = create_client()
    lcel_chain = build_lcel_chain()
    matched = 0
    for text in TEST_INPUTS:
        if compare(client, lcel_chain, text):
            matched = matched + 1
    print(f"{matched}/{len(TEST_INPUTS)} matched")


if __name__ == "__main__":
    main()
```

Expected output (wording varies):

```
Maria wants to reset her password -> match: True
  raw:  name='Maria' intent='reset her password'
  lcel: name='Maria' intent='reset her password'
Hi, I'm Tom and my invoice looks wrong -> match: True
  raw:  name='Tom' intent='check a wrong invoice'
  lcel: name='Tom' intent='check a wrong invoice'
Ahmed here, please cancel my order -> match: True
  raw:  name='Ahmed' intent='cancel his order'
  lcel: name='Ahmed' intent='cancel his order'
3/3 matched
```

Back to the exercise · Hint 1 · Hint 2 · Solution

Approach 2 — typed rows, `run_comparison()` split from `main()`

Story — `run_comparison()`: computing and reporting are split, and each result is a typed `ComparisonRow`. **If not:** the Build Task couldn't reuse the logic without copying the printing too.

```
# lcel_vs_raw_sdk_practice.py
from dataclasses import dataclass

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import Runnable
from langchain_openai import ChatOpenAI

from chat_client import EXTRACT_PROMPT, create_client, get_structured_reply
from schemas import ExtractedInfo

# how: every input has a name, so neither version returns None here
TEST_INPUTS = [
    "Maria wants to reset her password",
    "Hi, I'm Tom and my invoice looks wrong",
    "Ahmed here, please cancel my order",
]


@dataclass
class ComparisonRow:
    # why: one typed record per input, instead of printing inline — this
    # is what makes run_comparison()'s result reusable by a caller or a test
    input_text: str
    raw_result: ExtractedInfo
    lcel_result: ExtractedInfo
    matched: bool


def build_lcel_chain(model_name: str = "gpt-4o-mini") -> Runnable:
    prompt = ChatPromptTemplate.from_messages(
        [("system", EXTRACT_PROMPT), ("human", "{text}")]
    )
    model = ChatOpenAI(model=model_name, temperature=0)
    return prompt | model.with_structured_output(ExtractedInfo)


def run_comparison(test_inputs: list[str]) -> list[ComparisonRow]:
```

```
    # why: the logic (run both versions, record whether they matched) lives
    # here, separate from main()'s reporting — this is the shape the Build
    # Task's compare_with_raw_sdk.py copies almost unchanged
    client = create_client()
    lcel_chain = build_lcel_chain()
    rows = []
    for text in test_inputs:
        raw_result = get_structured_reply(client, text)
        lcel_result = lcel_chain.invoke({"text": text})
        # why: a name has one right answer; intent wording can differ
        matched = raw_result.name == lcel_result.name
        rows.append(ComparisonRow(text, raw_result, lcel_result, matched))
    return rows


def main() -> None:
    rows = run_comparison(TEST_INPUTS)

    matched = 0
    for row in rows:
        if row.matched:
            matched = matched + 1
    print(f"{matched}/{len(rows)} matched")
    for row in rows:
        if not row.matched:
            print(f"MISMATCH on: {row.input_text}")
            print(f"  raw:  {row.raw_result}")
            print(f"  lcel: {row.lcel_result}")


if __name__ == "__main__":
    main()
```

Expected output:

```
3/3 matched
```

Difference from Approach 1: Approach 1 mixes computing and printing in `compare()`. Approach 2 returns data from `run_comparison()` and reports in `main()` — the shape the Build Task's `compare_with_raw_sdk.py` copies almost unchanged.

Which one should you actually run? Approach 1 for a quick check. Approach 2 once you'll re-run the comparison after every prompt change or LangChain upgrade — it's what the Build Task uses.